AIVAR

INVOICES TESTING AGENT

TECHNICAL APPROACH DOCUMENT

Invoices Testing Agent

An AI-assisted quality-assurance platform that validates the data an AWS Lambda function extracts from freight invoices — comparing it against source PDFs and vendor mapping sheets, scoring accuracy, and suggesting prompt improvements.

PREPARED FOR	**Aivar Engineering**
SUBJECT SYSTEM	**Invoices Testing Agent (application repository)**
DOCUMENT DATE	**31 August 2026**
BASIS	**Direct review of the full backend and frontend source tree**
VERSION	**1.2**

Table of Contents

1. Purpose & Scope

This document describes the technical approach behind the **Invoices Testing Agent**, covering its architecture, component design, data model, and operating behavior exactly as implemented in the current codebase. It also provides the practical instructions needed to install, run, and use the application.

The content is derived directly from the source code — the FastAPI backend, the three AI agents, the DynamoDB schema, and the React frontend — rather than from planning documents. Where the codebase contains a proposed-but-not-yet-built capability (such as the AWS serverless migration), this is explicitly labelled as **proposed** and kept separate from the description of the system as it runs today.

IN SCOPE

Backend service design, the AI validation pipeline, data model, frontend application, AWS integrations, security model, local installation, and day-to-day usage.

OUT OF SCOPE

The internal implementation of the external Invoice Processor Lambda (the system under test) is out of scope — this document treats it as an upstream producer of CloudWatch logs and, optionally, of webhook pushes to this application.

2. Executive Summary

The Problem

The organization operates an AWS Lambda function that reads freight invoices (PDF/email attachments) and uses an LLM to extract structured data — invoice numbers, amounts, charge codes, dates, addresses — which it then posts to an internal API. If the extraction is wrong, billing errors propagate downstream. Manually checking every invoice against its source document does not scale.

The Solution

The Testing Agent is an automated QA layer that sits alongside the Lambda. For every invoice the Lambda processes, it:

1. Retrieves the data the Lambda extracted (either by reading CloudWatch logs, or by receiving it directly via a webhook the Lambda calls).
2. Retrieves the actual invoice PDF and the vendor's field/charge mapping spreadsheet from S3.
3. Uses an LLM (Claude, via AWS Bedrock) to compare the extracted payload against the PDF and mapping rules field-by-field, classifying each as `correct`, `wrong`, or `missing`.
4. Computes a weighted 0–100 score, applies a mandatory-field override, and generates specific suggestions for improving the Lambda's extraction prompt.
5. Persists the result to DynamoDB and surfaces it in a React dashboard for the team to review trends, drill into field-level detail, and track prompt-quality regressions over time.

Key Concepts

TERM	MEANING IN THIS SYSTEM
`Project`	A named configuration linking a CloudWatch log group, S3 mapping files, scoring weights, and mandatory fields for one carrier/workflow.
`Test Run`	One execution of the validation pipeline for a project — may cover one or several invoices found in the log window.
`Invoice Payload`	The JSON object the Lambda extracted from an invoice.
`Field Mapping`	A vendor-specific Excel sheet defining expected fields, extraction logic, and example values.
`Charge Map`	A shared Excel sheet mapping charge codes to master names and per-vendor charge names.
`Mandatory Field`	A field that, if absent, forces the overall test result to `failed` regardless of score.
`Pull Mode`	A user clicks "Run Test"; the backend fetches logs and files itself.
`Push Mode`	The Lambda posts its result directly to this backend's `/api/intake` webhook.

3. System Architecture

The system is a two-tier web application (React SPA + FastAPI service) backed by Amazon DynamoDB, with the AI reasoning delegated to AWS Bedrock and reference data pulled from S3 and CloudWatch Logs.

Figure 1 — Component and data-flow overview

Design principle: two independent entry points, one shared pipeline

Both operating modes converge on the same core logic: `run_input_collector` (fetch reference files) and `run_payload_validator` (LLM scoring). Pull mode additionally runs `run_log_analyzer` to discover invoices from CloudWatch; push mode skips this because the Lambda already supplies the payload directly in the webhook body. Both modes write to the same `results` collection and update the same project status fields, so the dashboard is agnostic to which mode produced a given result.

4. Technology Stack

Backend

COMPONENT	TECHNOLOGY	ROLE IN THIS SYSTEM
Web framework	`FastAPI`	Async REST API, automatic OpenAPI docs at `/docs`
Server	`Uvicorn`	ASGI server for local/dev execution
Agent framework	`Strands Agents SDK`	Tool-calling agent runtime used by the Log Analyzer and Payload Validator
LLM runtime	`AWS Bedrock (BedrockModel)`	Hosts the Claude model invoked by both AI agents
AWS SDK	`boto3`	S3, CloudWatch Logs, and Bedrock clients
Database	`Amazon DynamoDB (boto3)`	Serverless NoSQL store, accessed via the same boto3 session used for S3/CloudWatch/Bedrock — no separate driver or connection string
Excel parsing	`openpyxl`	Reads vendor field-mapping and charge-map workbooks
PDF parsing	`PyMuPDF (fitz)`	Converts invoice PDFs to Markdown as LLM-ready ground truth
Auth	`python-jose`	HS256 JWT issuance and verification
Config	`python-dotenv`	Loads `backend/.env` into process environment

Frontend

COMPONENT	TECHNOLOGY	ROLE IN THIS SYSTEM
UI library	`React 18`	Component-based single-page application
Build tool	`Vite 5`	Dev server (port 5173) with API proxy; production bundler
Routing	`React Router v6`	Client-side route guarding via JWT presence
Styling	`Tailwind CSS`	Utility-first styling with a custom brand color palette
Charts	`Recharts`	Score-trend line chart on the Results page
Icons	`Lucide React`	Icon set used throughout the UI

5. Backend Design

5.1 Directory Layout

```
backend/
├── main.py                  FastAPI app, CORS, router registration, lifespan hooks
├── config.py                Settings from .env + boto3 session factory
├── database.py               DynamoDB resource/client, table helpers, TTL attribute setup
├── agent_runner.py           Background-thread runner for pull-mode test runs
├── seed.py                   Inserts demo projects/results on first empty-DB start
│
├── agents/                   Pipeline logic
│   ├── orchestrator.py       Coordinates a pull-mode run across all invoices found
│   ├── log_analyzer.py       CloudWatch -> list of invoice analyses (3-tier fallback)
│   ├── input_collector.py    S3 download + Excel/PDF parsing (no AI)
│   ├── payload_validator.py  Bedrock/Claude agent — field-by-field scoring
│   └── intake_processor.py   Push-mode pipeline (webhook-driven, skips log analyzer)
│
├── routers/                  HTTP endpoints
│   ├── auth.py  projects.py  results.py  jobs.py  intake.py
│
├── models/                   Pydantic schemas
│   ├── project.py  result.py  auth.py  intake.py
│
├── services/                 Thin AWS/file-format wrappers, no AI
│   ├── s3.py  cloudwatch.py  excel_parser.py  pdf_parser.py
│
└── tools/                    @tool-decorated functions callable by Strands agents
    ├── dynamodb_tools.py  cloudwatch_tools.py  s3_tools.py
```

5.2 Configuration

All configuration is centralized in `backend/config.py` as a `Settings` class reading from environment variables (loaded via `python-dotenv` from `backend/.env`). A `make_aws_session()` factory returns a `boto3.Session` built from the configured AWS keys, falling back to the default credential chain (IAM role / `~/.aws/credentials`) when the keys are blank.

5.3 REST API Reference

All application routes are mounted under the `/api` prefix in `main.py`. Two unprefixed routes (`/` and `/health`) exist for liveness checks.

METHOD	PATH	AUTH	PURPOSE
POST	`/api/auth/login`	None	Exchange email + password for a 24h JWT
GET	`/api/auth/me`	JWT	Return the authenticated user's identity
GET	`/api/projects`	JWT	List all project configurations
POST	`/api/projects`	JWT	Create a project (409 if `project_id` exists)
GET	`/api/projects/{id}`	JWT	Fetch one project
PUT	`/api/projects/{id}`	JWT	Partially update a project

METHOD	PATH	AUTH	PURPOSE
DELETE	`/api/projects/{id}`	JWT	Delete a project and cascade-delete its results and jobs
GET	`/api/projects/{id}/results`	JWT	List results, filterable by invoice substring, status, carrier
GET	`/api/projects/{id}/carriers`	JWT	Distinct `vendor_name` values seen for the project
GET	`/api/results/{result_id}`	JWT	Fetch one test result in full detail
POST	`/api/projects/{id}/run-test`	JWT	Start a pull-mode test run; returns `{job_id}` (202)
GET	`/api/jobs/{job_id}/status`	JWT	Poll job progress/result (5 named steps)
POST	`/api/intake`	`X-Intake-Key`	Webhook — Lambda pushes invoice data (202, fire-and-forget)
GET	`/api/intake/health`	None	Liveness check for the intake tunnel

5.4 Data Models (Pydantic)

ProjectConfig (`models/project.py`) — `project_id` , `project_name` , `cloudwatch_log_group` , `email_recipients[]` , `target_api_url` , `file_slots[]` (each a bucket/key pair with `enabled` / `required` flags), `scoring_weights` (charge/address/date/amount, default 25 each), `log_window_hours` (default 24), `mandatory_fields[]` , `status` (`incomplete` / `configured` / `never_tested`), `last_tested` , `last_score` .

TestResult (`models/result.py`) — `result_id` , `project_id` , `invoice_number` , `timestamp` , `overall_score` , `status` (`passed` / `warning` / `failed`), `vendor_name` , `api_status` , `field_validations[]` (each with `expected_value` , `actual_value` , `status` , `source_used`), `prompt_suggestions[]` , `log_summary` (errors/warnings/duration/cold_start), `raw_payload` .

IntakePayload (`models/intake.py`) — the contract the Lambda must satisfy when calling `/api/intake` : routing keys (`project_id` / `s3_bucket` / `log_group` — at least one required), `invoice_number` , `payload` (required), plus optional `llm_response` , `prompt` , `execution_duration_ms` , `cold_start` , `errors` , `warnings` , `api_status` .

5.5 DynamoDB Schema

Persistence uses three DynamoDB tables in on-demand (pay-per-request) capacity mode, accessed through a single boto3 `dynamodb` resource shared with the rest of the app's AWS session. Each table uses a simple primary key plus, where the access pattern requires it, one Global Secondary Index (GSI).

TABLE	PRIMARY KEY	GSI	PURPOSE
`projects`	`PK: project_id`	—	One item per configured project; always fetched by exact `project_id` , so no secondary index is needed.
`results`	`PK: result_id`		

TABLE	PRIMARY KEY	GSI	PURPOSE
		`project_id-timestamp-index` (PK: `project_id`, SK: `timestamp`)	One item per invoice per test run. The GSI supports "list results for a project, most recent first" via `Query` with `ScanIndexForward=false`; the direct table key supports fetching a single result by ID.
`jobs`	`PK: job_id`	—	Tracks in-flight/recent test and intake jobs. A native DynamoDB **TTL attribute** (`expires_at`, epoch seconds, set to `created_at + 3600`) causes DynamoDB to auto-delete each item roughly one hour after creation.

Filtering without an aggregation pipeline

DynamoDB has no server-side aggregation framework, so the query-shaped filters the UI exposes are implemented differently than a document database would implement them:

- **Status / carrier / invoice-number filters** (`GET /api/projects/{id}/results`) — the backend `Query`s the `project_id-timestamp-index` GSI for the project, then applies a DynamoDB `FilterExpression` (exact match on `status` / `vendor_name`, `contains()` on `invoice_number`) to the page of items returned. This filters after the read rather than before it — at this system's expected per-project result volume that trade-off is acceptable, but it does not reduce read-capacity consumption the way a purpose-built index would.
- **Distinct carriers** (`GET /api/projects/{id}/carriers`) — rather than a MongoDB-style `$group` aggregation, the backend `Query`s all result items for the project via the same GSI and de-duplicates `vendor_name` values in application code.

Table creation is idempotent and happens once at startup via `ensure_tables()` (calling `create_table` for each table if it does not already exist, then waiting for `ACTIVE` status), invoked from the FastAPI `lifespan` handler in `main.py`. On first boot with empty tables, `seed.py` inserts six demo projects and four demo results so the UI is populated before any real project is configured.

6. The AI Agent Pipeline

Three agents cooperate to turn raw log/webhook data into a scored result. Two call an LLM; one is pure rule-based file I/O.

6.1 Orchestrator (`agents/orchestrator.py`) — pull mode only

Loads the project config, calls the Log Analyzer to discover every invoice in the configured CloudWatch window, then for each invoice: fetches the matching S3 reference files (keyed by `vendor_reference_id`), fetches and converts the invoice PDF, and either short-circuits to a failed result (if the Lambda's own API call returned HTTP ≥ 400) or runs the Payload Validator. Every invoice's result is persisted independently; the function returns the first result to the caller for the job-status response. It also exposes `find_project_by_bucket()`, matching a project by S3 bucket name with an exact-then-fuzzy (`difflib.SequenceMatcher`, threshold 0.75) strategy.

6.2 Log Analyzer (`agents/log_analyzer.py`)

Three-tier resolution strategy, in order:

1. **Rule-based direct parser** (`services/cloudwatch.py: extract_invoice_blocks`) — scans raw log messages for JSON objects, classifies them by *structure* (never by hardcoded message text): a transaction-tracking block has `transaction_id` plus a step/status key and is skipped; an object matching ≥ 2 of a broad set of invoice-field names is treated as invoice data. HTTP status codes, error/warning lines, execution duration, and cold-start flags are extracted from surrounding plain-text log lines using generic regex patterns (no message-string coupling).
2. **LLM fallback** — if the rule-based parser finds nothing, a Strands `Agent` bound to four CloudWatch tools and Claude (via Bedrock) is asked to read the raw logs and return the same structured JSON array.
3. **Mock data** — if both fail (e.g. no log group configured, or CloudWatch unreachable), a hardcoded sample invoice matching the real production payload shape is returned, so the pipeline and UI remain exercisable in local development without AWS access.

6.3 Input Collector (`agents/input_collector.py`) — no AI

For every enabled file slot on the project, downloads the S3 object. Excel files (`.xlsx` / `.xls`) are parsed via `services/excel_parser.py` into structured JSON: `parse_field_mapping()` opens the sheet named after the vendor reference ID and reads columns matched by header keyword (*Column Name*, *Ingestion Logic*, *Mapping Required*, *Remarks*, *Value Present*); `parse_charge_mapping()` opens the "Charge Map" sheet, where row 0 holds vendor-ID column headers and rows 2+ hold `charge_code / master_name / vendor_name` triples. Separately, `collect_invoice_pdf()` reads `payload.custom.attachment_bucket/attachment_key`, downloads the PDF, and converts it to Markdown via PyMuPDF — this becomes the ground-truth text handed to the validator.

6.4 Payload Validator (`agents/payload_validator.py`) — the core AI agent

A Strands `Agent` bound to a Bedrock Claude model and two tools (`compare_field_values` , `calculate_weighted_score`) receives, in a single prompt: the project's scoring weights and mandatory-field list, the Lambda's own captured prompt text (scanned for embedded charge rules), the invoice PDF Markdown (ground truth), the parsed Excel field/charge mappings, and the raw extracted payload. It is instructed to prioritize sources in this order:

1. Invoice PDF content (definitive ground truth — overrides mapping-file hints on conflict)
2. Excel field-mapping sheet
3. Excel charge-mapping sheet
4. Charge rules embedded in the Lambda's own prompt text

Container-level fields (nested under `payload.shipments[].container[]`) are only validated when at least one container entry exists — an empty/absent container array is not treated as a missing field. The agent returns a JSON object of per-field validations, an overall score, a status, and a list of specific prompt-improvement suggestions. A post-processing step (`_enforce_mandatory`) then tags each validation with `is_mandatory` and forces `status = "failed"` if any mandatory field is missing or incorrect, independent of the numeric score.

> **FALLBACK BEHAVIOR**
>
> If the Bedrock call raises an exception, `_fallback_validation()` runs instead: a rule-based check that only confirms whether known/mandatory fields are *present* (no expected-value comparison, no PDF or Excel cross-checking), with a single generic suggestion. This keeps the pipeline operational during Bedrock outages, at reduced validation fidelity.

6.5 Scoring Formula

```
overall_score =  (charge_fields_score  × charge_weight)
               + (address_fields_score × address_weight)
               + (date_fields_score    × date_weight)
               + (amount_fields_score  × amount_weight)

category_score = (fields marked "correct" in category) / (total fields in category)
                  − categories with zero matched fields default to full weight

status:  overall_score >= 85  ->  passed
         overall_score >= 60  ->  warning
         overall_score <  60  ->  failed

override: ANY mandatory field missing/incorrect  ->  status forced to "failed"
```

The default weight split is 25/25/25/25 across charge, address, date, and amount fields; each project can override this via its `scoring_weights` . Weights are relative — the UI does not require them to sum to 100.

7. Two Operating Modes

7.1 Pull Mode — triggered from the UI

```
Browser --POST /api/projects/{id}/run-test--> FastAPI --202 Accepted + job_id--> Browser
                                                  |
                                       agent_runner.start_test_run()
                                       spawns a daemon background thread
                                                  |
                                       orchestrator.run_test(project_id)
                                          1. Log Analyzer   -> list of invoices
                                          2. Input Collector -> S3 mapping + PDF (per invoice)
                                          3. Payload Validator -> score + suggestions
                                          4. save_test_result() + update_project_last_tested()
                                                  |
                                  DynamoDB job item updated with 5 named steps,
                                  final status, result_id and overall_score
```

The job document exposes 5 named progress steps (*Collecting inputs from S3 → Querying CloudWatch logs → Validating payload fields → Computing scores and suggestions → Saving result to database*), intended to be polled via `GET /api/jobs/{job_id}/status`.

7.2 Push Mode — the Lambda calls the webhook directly

```
AWS Lambda --POST /api/intake  (header: X-Intake-Key)--> FastAPI --202 Accepted + job_id--> Lambda
     body: { project_id | s3_bucket | log_group,  invoice_number, payload,
             llm_response, prompt, execution_duration_ms, cold_start,
             errors, warnings, api_status }
                                                  |
                                       background thread: intake_processor.process_intake()
                                          1. resolve_project()  — project_id -> s3_bucket -> log_group
                                          2. if api_status >= 400: record failed result, skip AI
                                          3. else: Input Collector + Payload Validator (as above)
                                          4. save_test_result(), tagged  source = "lambda_push"
```

Push mode skips the Log Analyzer entirely because the Lambda already supplies the payload; project resolution tries `project_id` (exact), then `s3_bucket` (exact match against any file slot's bucket, then fuzzy match at a 0.75 similarity threshold), then `cloudwatch_log_group` (exact). The intake endpoint is intentionally authenticated with a shared secret header rather than a JWT, since the Lambda has no user session.

8. Frontend Design

8.1 Routes

ROUTE	ACCESS	PAGE
`/login`	Public	Email/password form; on success stores JWT + user in `localStorage`.
`/`	JWT required	Dashboard — summary stat cards and a grid of project cards.
`/projects`	JWT required	Project list.
`/project/:id/configure`	JWT required	4-step configuration wizard (use `:id = "new"` to create).
`/project/:id/results`	JWT required	Filterable, expandable results table with score trend chart.
`/settings`	JWT required	All-projects management table (edit / view results / delete).

Route protection is a simple client-side guard (`PrivateRoute` in `App.jsx`) that checks for a token in `localStorage` and redirects to `/login` otherwise; the API client additionally clears the session and redirects on any `401` response.

8.2 The Configure Wizard

Implemented in `pages/Configure.jsx` as four steps: **1. Basic Info** (project name/ID, CloudWatch log group, target API URL), **2. Source Files** (toggle S3 bucket/key pairs for the Field Mapping Sheet, Charge Mapping, and Country Code Mapping slots, plus add/remove arbitrary custom mapping slots), **3. Test Config** (mandatory-fields tag input and four scoring-weight sliders), and **4. Review & Save** (read-only summary with "Save Project" or "Save & Run Test"). The prompt template and LLM-response sample are collected automatically from CloudWatch by the Log Analyzer and are not exposed as configurable file slots in this step.

8.3 The Results Page

Each result row is collapsible into four tabs: **Field Validation** (per-field table with expected/actual/status/source, a mandatory-fields summary bar, and a dedicated error state when the Lambda's own API call failed), **Prompt Suggestions** (copyable list), **Log Summary** (errors, warnings, duration, cold start), and **Raw Payload** (JSON tree viewer). A carrier filter, status filter, invoice-number search, and a Recharts score-trend line chart sit above the list.

8.4 API Client

`src/api/client.js` wraps `fetch` : it attaches `Authorization: Bearer <token>` from `localStorage` to every request, redirects to `/login` and clears the session on a `401` , and throws the backend's `detail` message on any other non-2xx response.

9. AWS Service Integration

SERVICE	USED BY	IAM ACTIONS NEEDED	PURPOSE
Amazon S3	`services/s3.py`	`s3:GetObject`, `s3:HeadObject`	Read vendor Excel mapping workbooks and invoice PDFs.
CloudWatch Logs	`services/cloudwatch.py`	`logs:StartQuery`, `logs:GetQueryResults`, `logs:DescribeLogGroups`	Read the invoice processor Lambda's log stream to discover invoice runs.
AWS Bedrock	`agents/log_analyzer.py`, `agents/payload_validator.py`	`bedrock:InvokeModel`	Runs the Claude model (`BEDROCK_MODEL_ID`, default `us.anthropic.claude-sonnet-4-6`) for both LLM-backed agents.
Amazon DynamoDB	`database.py`, `tools/dynamodb_tools.py`	`dynamodb:GetItem`, `PutItem`, `UpdateItem`, `DeleteItem`, `Query`, `DescribeTable`, `CreateTable`	Reads/writes the `projects`, `results`, and `jobs` tables described in §5.5.

Credentials are resolved by `config.make_aws_session()`: explicit `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` from `.env` if present, otherwise boto3's default credential chain (instance/IAM role, or `~/.aws/credentials`).

10. Security Model

10.1 UI Authentication

A single administrator account is hardcoded in `routers/auth.py` (`admin@aivar.tech` / `admin@123`) — there is no user table or per-user credential store. A successful login returns an HS256 JWT (24-hour expiry, secret from `JWT_SECRET_KEY`) which every protected endpoint validates via the `get_current_user` FastAPI dependency.

10.2 Lambda Webhook Authentication

`/api/intake` is deliberately not JWT-protected — the Lambda has no user session. It instead requires a shared-secret header, `X-Intake-Key` , compared against `INTAKE_API_KEY` . This keeps the webhook independent of the human login system.

10.3 CORS

`main.py` allows only `http://localhost:5173` and `http://127.0.0.1:5173` by default — the Vite dev server origins. A production deployment must add its real frontend origin to `allow_origins` .

SECURITY CONSIDERATIONS FOR PRODUCTION

- Login credentials and both secrets (`JWT_SECRET_KEY` , `INTAKE_API_KEY`) are currently simple defaults suitable for local development only — they must be rotated to strong, randomly generated values before any non-local deployment.
- There is no per-user account model, role separation, or audit trail beyond application logs.
- AWS access keys, if used instead of an IAM role, are stored in a plaintext `.env` file.

11. Implementation Notes & Known Limitations

These are verified, code-level observations worth carrying into any roadmap discussion:

- **The "Run Test" progress modal does not poll the real job-status endpoint.** `RunTestModal.jsx` fires `POST /run-test` once, then animates a fixed six-step progress sequence with client-side `setTimeout` calls and shows `project.last_score ?? 91` as the "final" score — it does not call `GET /api/jobs/{job_id}/status` (that function exists in `api/client.js` but is currently unused). The authoritative result only appears after navigating to or refreshing the Results page.
- **Dashboard summary stats are partly static.** In `Dashboard.jsx`, "Tests Run Today", "Average Score", and "Failed Tests" are hardcoded placeholder values, not computed from live data; only "Total Projects" reflects the real project count.
- **`ANTHROPIC_API_KEY` is defined but unused.** `config.py` reads this variable, but no code path in the repository calls the Anthropic API directly — both AI agents route inference through AWS Bedrock via the Strands `BedrockModel`.
- **Log analysis and demo data include realistic fallbacks.** When CloudWatch or Bedrock is unreachable, or no log group is configured, the system deliberately falls back to mock invoice data or rule-based checks so the UI remains usable in local development — this is by design, not a defect, but it means a "successful" test run in a fresh local environment may reflect fallback data rather than a live AWS integration.
- **Single hardcoded admin account.** See §10.1 — there is currently no multi-user support.

12. Deployment Approach

12.1 Current — Local Development

The application runs today as two local processes: `uvicorn main:app --port 3001` for the backend and `vite` (port 5173) for the frontend, with Vite proxying `/api/*` to the backend to avoid CORS issues (see `frontend/vite.config.js`). Persistence is a set of Amazon DynamoDB tables in the configured AWS region, reached through the same AWS credentials already used for S3, CloudWatch, and Bedrock; no separate database connection string or credential type is needed. For fully offline development, `DYNAMODB_ENDPOINT_URL` can point the same code at a local DynamoDB Local container instead of the real AWS endpoint.

12.2 Proposed — AWS Serverless Migration (not yet implemented)

> **STATUS: DOCUMENTED PLAN ONLY**
>
> The repository's `AWS_DEPLOYMENT_GUIDE.md` lays out a serverless target architecture, but as of this review **none of its code changes exist in the codebase** — there is no `lambda_handler.py`, no `sqs_worker.py`, and no `Dockerfile` in `backend/`. It is included here as the intended future direction, not as a description of a working deployment.

The proposed design would replace the backend's in-process background threads (which cannot survive past an API Gateway response, and cannot run for the full duration a Bedrock-driven validation may take) with:

- **API Lambda** — the existing FastAPI app wrapped by `Mangum`, fronted by an API Gateway HTTP API, timeout 30s.
- **SQS queue** — decouples the instant API response from long-running validation work.
- **Worker Lambda** — triggered by SQS, runs the Strands orchestrator with a 15-minute timeout.
- **Secrets Manager** — replaces the `.env` file for production secrets.
- **S3 + CloudFront** — static hosting for the built React frontend.
- **ECR** — container images for both Lambda functions (PyMuPDF's native dependencies require a container image rather than a zip package).

S3 reference-file storage and CloudWatch logging require no changes under this plan. DynamoDB is a natural fit for the Lambda-based target architecture: unlike a self-hosted or Atlas-style cluster, it needs no VPC configuration, no PrivateLink, and no IP allow-listing for Lambda's dynamic addresses — the Worker and API Lambdas reach it through the same IAM role already granted S3/CloudWatch/Bedrock access. Estimated infrastructure cost for low-to-medium volume (thousands of tests/month) is roughly $3–6/month including on-demand DynamoDB usage. Full step-by-step provisioning commands for the remaining serverless components are maintained in `AWS_DEPLOYMENT_GUIDE.md` at the repository root.

13. Installation & Setup Guide

13.1 Prerequisites

- Python 3.11 or newer
- Node.js 18 or newer
- AWS credentials with S3, CloudWatch Logs, Bedrock, and DynamoDB permissions (or an assumable IAM role)
- Optional: Docker, if you want to run DynamoDB Local for fully offline development instead of a real AWS DynamoDB endpoint

13.2 Step 1 — Configure environment variables

```
cp backend/.env.example backend/.env
```

Edit `backend/.env`:

```
DYNAMODB_TABLE_PREFIX=invoices_testing_agent
# Leave DYNAMODB_ENDPOINT_URL blank for real AWS.
# Set it to http://localhost:8000 to target DynamoDB Local instead.
DYNAMODB_ENDPOINT_URL=
JWT_SECRET_KEY=change-me-in-production
AWS_REGION=ap-south-1
AWS_ACCESS_KEY_ID=your-aws-access-key
AWS_SECRET_ACCESS_KEY=your-aws-secret-key
LOG_GROUP_PREFIX=/aws/lambda/invoice-processor
INTAKE_API_KEY=your-shared-secret-with-lambda
```

`DYNAMODB_TABLE_PREFIX` is used to derive the three table names (`{prefix}_projects`, `{prefix}_results`, `{prefix}_jobs`), keeping multiple environments (dev/staging/prod) isolated within the same AWS account.

13.3 Step 2 — Install dependencies

```
# Backend
cd backend
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# Frontend
cd ../frontend
npm install

# Or, from the repository root, install both at once:
make install
```

13.4 Step 3 — Start the servers

```
# Terminal 1 — backend (port 3001)
cd backend && source .venv/bin/activate
uvicorn main:app --reload --port 3001

# Terminal 2 — frontend (port 5173)
cd frontend
npm run dev

# Or, from the repository root:
make dev
```

13.5 Step 4 — Verify startup

A healthy backend prints a DynamoDB table check — it verifies the three tables exist (creating any that are missing) and waits for them to reach `ACTIVE` status — followed by the Uvicorn startup banner. A `✗ FAILED` line indicates the configured AWS credentials lack DynamoDB permissions, or `AWS_REGION` / `DYNAMODB_ENDPOINT_URL` point somewhere unreachable.

13.6 Step 5 — Open the app

Visit `http://localhost:5173` and sign in with the built-in credentials:

Email	`admin@aivar.tech`
Password	`admin@123`

13.7 Quick Reference

Install everything	`make install`
Start backend only	`make dev-backend`
Start frontend only	`make dev-frontend`
Start both together	`make dev`
Backend URL	`http://localhost:3001`
Frontend URL	`http://localhost:5173`
Interactive API docs	`http://localhost:3001/docs`

14. User Guide — How to Use the Application

14.1 Signing in

Open `http://localhost:5173/login` and sign in with the credentials in §13.6. On success you land on the Dashboard; the session persists in the browser until you sign out or the 24-hour token expires.

14.2 Creating and configuring a project

1. From the Dashboard or Projects page, click **New Project**.
2. **Step 1 — Basic Info:** enter a Project Name (the Project ID auto-generates as a slug — lowercase, hyphenated), and the CloudWatch Log Group the target Lambda writes to (e.g. `/aws/lambda/invoice-processor-ge-freight`). These three fields are required before you can continue.
3. **Step 2 — Source Files:** toggle on any of the three default S3 mapping slots (Field Mapping Sheet, Charge Mapping, Country Code Mapping) and supply their bucket + key, or add a custom mapping slot. Leave a slot disabled if that reference file does not apply to this project — the prompt template and LLM response are collected automatically from CloudWatch and need no configuration here.
4. **Step 3 — Test Config:** type mandatory field names (press Enter or comma after each) that must always be present — a missing mandatory field forces the result to *failed* regardless of score. Adjust the four scoring-weight sliders (charge/address/date/amount) to reflect what matters most for this carrier.
5. **Step 4 — Review & Save:** confirm the summary, then either **Save Project** (returns to the Dashboard) or **Save & Run Test** (saves and immediately opens the Run Test dialog).

14.3 Running a test

From a project's Results page, or immediately after saving a new project, open **Run Test**. Optionally enter an invoice-number filter to target one specific invoice; leave it blank to process every invoice found in the configured log window. Click **Start Test** — this posts to `/api/projects/{id}/run-test` and the backend begins fetching logs and reference files in the background.

> **WHERE TO CHECK THE REAL RESULT**
>
> The in-modal progress animation is illustrative (see §11) — after it finishes, click **View Results** or navigate to the project's Results page and use **Refresh** to load the freshly saved result(s) from the database.

14.4 Reviewing results

On the Results page, use the search box, status tabs (All/Passed/Warning/Failed), carrier pills, and date-range selector to narrow the list. Click any result row to expand it into four tabs:

- **Field Validation** — per-field expected vs. actual values, a mandatory-fields status bar, and (when the Lambda's own API call failed) a dedicated error banner instead of field data.
- **Prompt Suggestions** — copyable, numbered improvement suggestions for the Lambda's extraction prompt.
- **Log Summary** — errors, warnings, execution duration, and cold-start flag captured from CloudWatch.

- **Raw Payload** — the full extracted JSON in a collapsible tree viewer.

14.5 Managing projects (Settings page)

The Settings page lists every project in a single table with quick actions: **Edit Config** (reopens the wizard), **Results** (jumps to that project's results), and a delete action (with inline confirmation) that removes the project and cascades to delete all of its stored results and jobs.

14.6 Wiring up a Lambda for push mode (for the Lambda team)

Instead of relying on the UI's pull-mode "Run Test", the invoice processor Lambda can push its result directly after every invoice run:

```
POST /api/intake
Headers:
  Content-Type: application/json
  X-Intake-Key: <value of INTAKE_API_KEY>
Body (application/json):
{
  "project_id": "ge-freight",          // or supply s3_bucket / log_group instead
  "invoice_number": "INV-2024-0091",
  "payload": { ... extracted invoice JSON ... },
  "llm_response": { ... },              // optional
  "prompt": "...",                       // optional — the prompt sent to the LLM
  "execution_duration_ms": 3240,
  "cold_start": false,
  "errors": [], "warnings": [],
  "api_status": 200
}
```

The endpoint responds `202 Accepted` immediately with a `job_id`; the Lambda should use a short client-side timeout (a few seconds) since validation continues asynchronously on the backend. Call `GET /api/intake/health` beforehand to confirm the endpoint is reachable.

15. Appendix A — Environment Variable Reference

VARIABLE	DEFAULT	PURPOSE
`DYNAMODB_TABLE_PREFIX`	`invoices_testing_agent`	Prefix used to derive the `projects` / `results` / `jobs` DynamoDB table names.
`DYNAMODB_ENDPOINT_URL`	`(blank)`	Blank targets real AWS DynamoDB in `AWS_REGION`; set to a local endpoint (e.g. `http://localhost:8000`) to target DynamoDB Local instead.
`JWT_SECRET_KEY`	`change-me-in-production`	HS256 signing key for login tokens. Rotate before any shared deployment.
`AWS_REGION`	`us-east-1`	Region for S3, CloudWatch, and Bedrock clients.
`AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY`	`(blank)`	Explicit AWS credentials; blank falls back to the default credential chain.
`LOG_GROUP_PREFIX`	`/aws/lambda/invoice-processor`	Reserved naming convention for Lambda log groups (informational).
`ANTHROPIC_API_KEY`	`(blank)`	Defined but not referenced by any current code path — see §11.
`INTAKE_API_KEY`	`change-me-intake-secret`	Shared secret required in the `X-Intake-Key` header on `/api/intake`.
`BEDROCK_MODEL_ID`	`us.anthropic.claude-sonnet-4-6`	Bedrock model ID used by both LLM-backed agents.

16. Appendix B — Command Reference

`make install`	Install backend (pip) and frontend (npm) dependencies.
`make dev`	Start backend and frontend concurrently.
`make dev-backend`	Start only `uvicorn main:app --reload --port 3001`.
`make dev-frontend`	Start only `vite` on port 5173.
`python backend/seed.py`	Manually (re-)run the demo data seed (no-op if collections are non-empty).

17. Appendix C — Glossary

`Bedrock`	Amazon's managed LLM hosting service; runs the Claude model used for reasoning.
`Strands Agents SDK`	The tool-calling agent framework used to build the Log Analyzer and Payload Validator.
`vendor_reference_id`	The field in an invoice payload used to select which Excel sheet/column applies to that carrier.
`Mandatory field`	A configured field whose absence forces the test result to `failed`.
`Pull mode`	The backend actively fetches CloudWatch logs and S3 files after a UI-triggered run.
`Push mode`	The Lambda proactively posts its extracted data to `/api/intake`.
`TTL attribute`	A DynamoDB item attribute (epoch timestamp) that causes DynamoDB to automatically delete the item once that time passes — used on the `jobs` table (~1 hour).
`GSI`	Global Secondary Index — an alternate partition/sort key view over a DynamoDB table, used here to query the `results` table by `project_id` and `timestamp`.

End of document. This technical approach document was compiled by direct inspection of the application's backend and frontend source code as of 31 August 2026, and supersedes narrative claims in the project's own architecture notes where the two differ (see §11).

Revision 1.1: the persistence layer described throughout this document (§5.5, §9, §12, §13, Appendix A) has been updated from the codebase's current MongoDB/PyMongo implementation to a target design on Amazon DynamoDB, per engineering direction. All other sections continue to reflect the codebase exactly as reviewed; the backend's `database.py` and `tools/mongodb_tools.py` modules still implement MongoDB today — the DynamoDB table names, GSI, and TTL-attribute design in this revision describe the intended replacement, not code that currently exists in the repository.

Revision 1.2: all product naming has been genericized to **Invoices Testing Agent** throughout this document. The reviewed codebase's actual hardcoded strings — the demo login (`pando@aivar.tech` / `pando@123`), the default database/table name prefix (`pando_testing_agent`), and UI text in the sidebar and login screen — still literally read "Pando" today; this revision does not change the source code, only how the system is named in this document.